

Sistemas Distribuídos e Tolerantes a Falhas

Relatório do Trabalho - Parte 1

Nomes:

Lucas Neto Ferreira (00597912)

Jean Smaniotto Argoud (00275602)

(A) Como foi implementado cada subserviço;

Subserviço de Descoberta - Na inicialização do cliente ele chama `descobrir_servidor()` para em broadcast enviar requisições do tipo `TIPO_DESCOBERTA` constantemente até que receba alguma resposta do servidor, que também vai estar em um loop ouvindo requisições na mesma porta. O servidor coloca o cliente na lista de clientes e o cliente guarda o endereço.

Subserviço de Processamento - Depois de estabelecida a conexão o cliente chama `enviar_com_confirmacao()` de acordo com o input do usuário e se o servidor não responder com confirmação ele reenvia a request. A confirmação do servidor vai voltar com os dados do servidor como a soma total e a contagem total de requests que o servidor recebeu até o momento.

Subserviço de Interface - No cliente `thread_leitura()` é onde lemos os inputs do usuário enquanto no servidor `log_servidor()` é que printa os dados no console.

(B) Em quais áreas do código foi necessário garantir sincronização no acesso a dados;

O mesmo cliente não vai ter alterações de suas variáveis paralelamente pois as suas variáveis que sofrem escrita ou não executam em threads diferentes ou são sequenciais na mesma thread. No servidor não há concorrência, ele funciona em uma única thread sequencial então não haverá conflito entre as requests dos clientes. A princípio nesta etapa do trabalho não foi especificado que o servidor precisaria ser multithread, mas caso ele fosse precisaria que antes entre a linha 38 e 48 estivessem em uma thread separada com lock entre elas.

(C) Descrição das principais estruturas e funções que a equipe implementou;

`FORMATO_PACOTE` - tipo de requisição, id de requisição, número de requests que o servidor recebeu até agora, somatório atual guardado no servidor.

`empacotar()` - Utiliza `FORMATO_PACOTE` para enviar uma request entre serviços.

`desempacotar()` - Recebe uma request do tipo `FORMATO_PACOTE`.

`descobrir_servidor()` - Envia request em broadcast para o servidor responder com seu endereço.

`enviar_com_confirmacao()` - Envia um número para o servidor somar em seu total, envia constantemente até receber resposta de confirmação.

`thread_leitura()` - Constantemente lê o que o usuário digita.

`log_cliente()` - Imprime no terminal do cliente o timestamp atual com uma mensagem.

`log_servidor()` - Imprime no terminal do servidor o timestamp atual com uma mensagem.

main() - Servidor fica ouvindo uma porta constantemente por qualquer request, respondendo seu endereço para as de descoberta e nas de requisição soma o valor recebido no total do servidor e responde todos os dados atuais para o cliente.
tabela cliente - guarda todos os clientes e o id de sua ultima request, assim como o total somado nela.

(D) Explicar o uso das primitivas de comunicação inter-processos usadas;

O sistema usa UDP, em um momento enviando em broadcast para o servidor e em outro já sabendo o endereço do servidor usamos um request-reply.

Problemas encontrados:

A princípio não tivemos grandes problemas, só ficamos em dúvida se pela definição do enunciado nesta etapa era exigido ou não usar multithread no servidor para processar simultaneamente várias requests de clientes, mas interpretamos que não seria necessário.